LegalEase

AI-Powered Legal Document Generator

Project Report

Submitted by: [Your Name]

Register / Roll No.: [Your Number]

Guide: [Guide Name]

Institution: [Your College / University]

Date: 28 September 2026

Abstract

Drafting a legal document such as a non-disclosure agreement, a rental agreement or an employment contract is slow and expensive for individuals and small businesses. Lawyers charge for routine work, free templates found online are generic, and people often sign documents they only partly understand. **LegalEase** is a web-based application, written in Python, that generates first drafts of common legal documents from a short form. The user chooses a document type, enters the parties and key terms, and receives a structured draft that can be edited in the browser and downloaded as a Word (.docx) or text file.

LegalEase works in two modes. In *template mode* it assembles the document from a library of standard clauses and fills in the user's details. In *AI mode* it sends the details to a large language model, Anthropic's Claude, through the Anthropic Messages API, which writes a tailored draft and can follow extra instructions such as adding a non-compete clause. If the AI service is unavailable, the application falls back to templates automatically. The whole application uses only the Python standard library, so it runs with no installation beyond Python itself. LegalEase produces drafts for review by a qualified lawyer; it does not provide legal advice.

1. Introduction

Legal agreements govern everyday life: renting a home, starting a job, hiring a freelancer or sharing business ideas. Most of these agreements follow well-known patterns, and the parts that change are the names, amounts, dates and a handful of special terms. Even so, producing a clean first draft normally means paying a professional or adapting an old document by hand, which invites mistakes such as a wrong party name, a missing clause or a forgotten governing-law statement.

Recent language models are good at producing structured, formal text, which makes them a natural fit for document drafting. Used carelessly, however, they can invent facts or produce unreliable clauses. LegalEase is therefore designed around two ideas: keep the user in control of the facts, and keep the output clearly marked as a draft that a human must review. Every generated document begins with a note that it is a draft for lawyer review, and the interface displays a permanent warning.

2. Problem Statement and Objectives

Problem statement. Individuals and small organisations need quick, affordable and consistent first drafts of routine legal documents, but existing options are either costly (lawyers), too generic (static templates) or unreliable when used without structure (unconstrained AI chat).

Objectives. The project aims to:

- provide a simple form-based interface for four common document types;
- generate complete, well-structured drafts with numbered clauses and a signature block;
- combine reliable template output with optional AI customisation;
- let users edit the draft and export it as .docx or .txt;
- degrade gracefully when the AI service or network is unavailable;
- avoid third-party dependencies so that the tool is easy to install and audit.

Scope. The current version supports a Non-Disclosure Agreement, a Rental / Lease Agreement, an Employment Contract and a Freelance / Service Agreement. It runs locally on the user's computer and serves a single user. Clause wording is general and defaults to an Indian (Tamil Nadu) jurisdiction, which the user can change.

3. Technologies Used

Component	Technology	Purpose
Language	Python 3.8+	All application logic; standard library only
Web server	http.server (ThreadingHTTPServer)	Serves the page and handles form submissions
Front end	HTML, CSS, JavaScript	Responsive form; fields change with document type
AI engine	Anthropic Messages API (Claude)	Optional tailored drafting in AI mode
Networking	urllib.request	Calls the AI API without external libraries
Export	zipfile + Office Open XML	Builds a valid .docx file from scratch

4. System Design

LegalEase follows a simple client-server design. The browser shows a form; the Python server receives the submitted values, chooses a generation method and returns the page with an editable draft. Downloads are handled by a separate route that converts the (possibly edited) text into the requested file format.

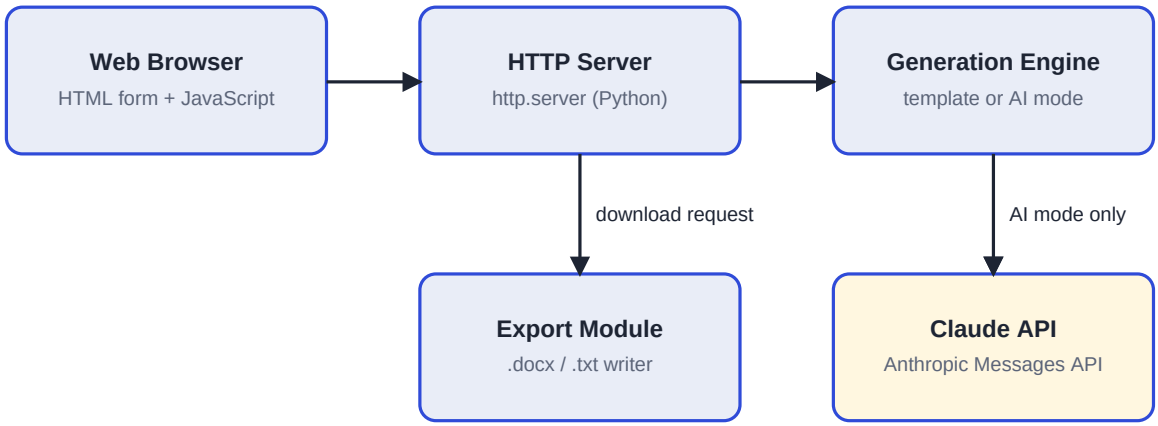

Figure 1: High-level architecture of LegalEase

4.1 Request flow

- The browser loads the page; JavaScript builds the input fields for the chosen document type.
- On *Generate*, the form is posted to `/generate`.
- The server validates the input and, in AI mode, builds a prompt from the labelled fields, the jurisdiction, the tone and any extra instructions, then calls the Claude API.

- If the API call fails for any reason, the server shows the error and produces the template draft instead, so the user always receives a usable result.
- The draft is returned in an editable text area. On *Download*, the current text is posted to `/download` and returned as a .docx or .txt attachment.

4.2 Design decisions

No external dependencies. Because everything uses the standard library, there is nothing to install and fewer components that can break or introduce security problems. Even the Word export is written by hand as a minimal Office Open XML package rather than depending on a document library.

Escaping and safety. All user-supplied text is HTML-escaped before it is placed on a page, and XML-escaped before it is written into a .docx file, which prevents injected markup from being interpreted. The server catches exceptions per request so a single bad request cannot stop it.

Human in the loop. The draft is editable, labelled as a draft and never presented as final legal advice.

5. Implementation

The program is a single file, `legalease.py`, organised into clear modules.

Module	Responsibility
Document catalogue	Defines each document type: its input fields with defaults, an introductory sentence, an ordered list of clauses and the names of the signing parties.
Template engine	Fills clause text with the user's values. Missing values appear as visible placeholders such as [TENANT] so nothing is silently invented.
AI engine	Builds the prompt and calls the Claude API over HTTPS. Handles HTTP errors, network errors and empty responses with clear messages.
DOCX writer	Converts the Markdown-style draft into headings, paragraphs and bullets and packages them as a valid Word file (content types, relationships, document body).
Web layer	Renders the page, parses form submissions and routes GET / and POST /generate and /download.

5.1 Template mode

Each clause is stored as a short piece of standard wording containing named placeholders. For example, the rent clause of the rental agreement states that the tenant shall pay rent of *{rent}* per month. At generation time the values entered by the user replace the placeholders, the jurisdiction is inserted into the governing-law clause, and today's date is added to the opening paragraph. The result is deterministic: the same inputs always give the same document, which makes template mode predictable and easy to test.

5.2 AI mode

In AI mode the server sends a system instruction and a user prompt to the model. The system instruction tells the model to use Markdown headings and numbered clauses, include a signature block, mark missing details with placeholders rather than inventing them, start with a draft-for-review note and output only the document. The user prompt lists the document type,

jurisdiction, tone (formal, plain English or balanced), the labelled details, the additional instructions and the minimum set of clauses required. The model name is configurable through an environment variable, and the API key is read from the environment and never stored in the code or the page.

5.3 Export

The Word export parses the draft line by line: lines beginning with # or ## become headings, hyphen lines become bullets, and italic markers are converted to italic text. The paragraphs are placed in `word/document.xml` together with the two small files that every .docx package requires. Signature lines made of underscores are preserved so the printed document can be signed by hand.

6. Testing and Results

The application was tested by starting the server and sending real HTTP requests to every route. The table summarises the checks and their outcomes.

Test	Expected behaviour	Result
Page load (GET /)	Returns the form with status 200	Passed
Generate, all 4 document types	Full draft returned for each type	Passed
Special characters (& and <>)	Escaped safely in the page	Passed
Rental draft with all fields	No unfilled placeholders remain	Passed
DOCX download	Valid file that opens in a Word library; 20 paragraphs; signature lines kept	Passed
TXT download	Plain-text attachment with correct name	Passed
Unknown route / empty POST	404 for unknown route; no server crash	Passed
AI mode with invalid key	Error shown; template draft displayed instead	Passed

These tests show that the core workflow (choose, generate, edit, download) works and that the application handles bad input and AI failures without crashing. A successful AI response was not verified in the test environment because no valid API key was available; the request follows the documented Messages API format, and the failure path was verified. The browser-side script that builds the input fields was also not exercised in an automated browser test. Both should be checked manually before deployment.

7. Limitations

- **Not legal advice.** Drafts are generic and may not comply with every local law or reflect a user's particular situation; lawyer review is essential.
- **AI reliability.** Language models can produce incorrect or inconsistent clauses, so AI output must be read carefully.
- **Limited catalogue.** Only four document types are supported, and template clauses are intentionally short.
- **Single user, local use.** There are no accounts, saved history or access control, and the built-in server is not intended for public internet deployment.

- **Basic formatting.** The Word export uses simple headings and paragraphs without styles, numbering fields or page numbers.

8. Future Enhancements

- More document types (sale deeds, partnership deeds, terms of service, privacy policies).
- Support for multiple jurisdictions with reviewed, jurisdiction-specific clause libraries and regional languages such as Tamil and Hindi.
- A clause-explanation feature that summarises each clause in plain language and highlights risky terms.
- PDF export, e-signature integration and secure cloud storage of drafts with user accounts.
- Deployment behind a production web framework (for example Flask or FastAPI) with authentication, logging and rate limiting.
- Automatic checks that the AI output contains every required clause and that no party name or amount differs from the form input.

9. Conclusion

LegalEase demonstrates that a small, dependency-free Python program can turn a short form into a well-structured legal draft in seconds. By combining dependable templates with optional AI customisation, and by keeping the user in control of the facts and the final text, the project offers a practical balance between speed and reliability. Its safeguards (visible draft warnings, placeholders for missing data, escaping of input and graceful fallback) show how AI assistance can be added to a sensitive domain responsibly. With a broader clause library, jurisdiction-specific review and stronger deployment, the same design could grow into a useful tool for individuals and small businesses who need quick access to first drafts.

References

- Python Software Foundation. *Python 3 Documentation: http.server, urllib, zipfile.* docs.python.org.
- Anthropic. *Claude API Documentation: Messages API.* docs.claude.com.
- ECMA International. *ECMA-376: Office Open XML File Formats* (WordprocessingML).
- Mozilla Developer Network. *HTML, CSS and JavaScript documentation.* developer.mozilla.org.